

Cryptographic Review of SauronID Differential Privacy and Homomorphic Encryption Modules

Date of original review: 2026-05-25 **Date of remediation addendum:** 2026-05-25 **Repository:** hackeurope-24, commit 7ac659e on branch `main` (original review); remediation applied post-review on same date **Reviewer:** [Reviewer Name], [Credential] **Engagement:** Independent cryptographic correctness review for enterprise compliance pack

1. Scope

The review covers the following source files in `core/src/` :

Subsystem	File
Differential privacy	<code>dp/laplace.rs</code>
Differential privacy	<code>dp/gaussian.rs</code>
Differential privacy	<code>dp/budget.rs</code>
Differential privacy	<code>dp/composition.rs</code>
Differential privacy	<code>dp/k_anonymity.rs</code>
Differential privacy	<code>dp/ledger.rs</code>
Aggregation pipeline	<code>aggregation/publish.rs</code>
Homomorphic encryption	<code>he/paillier.rs</code>
Homomorphic encryption	<code>he/encoding.rs</code>
Homomorphic encryption	<code>he/serde_impl.rs</code>
Aggregation pipeline	<code>aggregation/he_aggregator.rs</code>

The review does **not** cover: HTTP routing, authentication, transport security, secret storage outside the HE keystore, or the broader product surface.

2. Methodology

1. Static review of each file against the underlying primitives and theorems cited in the source comments.
2. Execution of the in-tree unit test suite for the four module groups (`dp::*` , `he::*` , `aggregation::publish` , `aggregation::he_aggregator`).
3. Cross-checking implemented formulae against:
4. Dwork & Roth, *The Algorithmic Foundations of Differential Privacy*, 2014.
5. Mironov, *Renyi Differential Privacy*, CSF 2017.

6. Paillier, *Public-Key Cryptosystems Based on Composite Degree Residuosity Classes*, EUROCRYPT 1999.
7. Balle & Wang, *Improving the Gaussian Mechanism for Differential Privacy: Analytical Calibration and Optimal Denoising*, ICML 2018.
8. Mironov, *On Significance of the Least Significant Bits for Differential Privacy*, CCS 2012.
9. Verification of the property-tests requested in the engagement scope.

Test results: 72 unit tests pass (`dp:: 32`, `he:: 28`, `aggregation::publish 9`, `aggregation::he_aggregator 3`). The encrypt-decrypt round trip, homomorphic addition, and scalar multiplication property tests requested in the scope are all present in `he/paillier.rs::tests` and pass.

3. Threat Model Under Which the Findings Are Certified

The guarantees in section 5 hold against an adversary who:

- has unbounded computation,
- observes every published output of the publication pipeline,
- knows the source code of the mechanisms and all public parameters (epsilon, delta, k, sensitivity, modulus),
- does **not** observe the input data,
- does **not** control the server's random number generator,
- does **not** have side-channel access to the server (no timing-precise CPU co-residency, no power analysis),
- in the homomorphic-encryption case, does **not** hold the Paillier private key.

Specifically excluded from the threat model:

- Operator-level administrators with access to the unsuppressed `customer_stats` table or to the `/v1/stats/cohort` raw endpoint. The DP guarantee applies only to *published* aggregates released through `publish_cohort_with_ledger`.
- Adversaries with the ability to time decryption operations at sub-microsecond resolution (`num-bigint::modpow` is not constant-time; see finding F-6).
- Malicious clients submitting forged ciphertexts to the HE aggregator (see finding F-3).

4. Findings

Severity rubric: - **High** — invalidates the privacy guarantee under the stated threat model. - **Medium** — weakens the guarantee, narrows the operating envelope, or expands the trust assumption beyond what is documented. - **Low** — robustness, code quality, or theoretical concern with negligible practical exposure.

F-1 (Medium-High) Gaussian sigma formula is only valid for epsilon less than or equal to 1

File: `dp/gaussian.rs:42-46`

The implemented formula `sigma = sensitivity * sqrt(2 * ln(1.25 / delta)) / epsilon` (Dwork-Roth 2014 eq. 3.8 / Theorem A.1 in the original paper) is **only proven to give (epsilon, delta)-DP for epsilon less than or equal to 1**. The constructor `GaussianMechanism::new` accepts any positive epsilon without warning.

For `epsilon > 1` the published bound does not hold. Operators wiring this mechanism in must either: - Constrain `epsilon ≤ 1` at the call site, or - Replace the formula with the analytic Gaussian mechanism (Balle-Wang 2018), which gives a tight calibration for all `epsilon > 0`.

The codebase currently uses Gaussian only via the RDP accountant (`composition::RdpAccountant::add_gaussian`), where the operating envelope is bounded by RDP composition rather than direct (epsilon, delta) sampling. The standalone `GaussianMechanism` is at present unused in the publication path, but it is a public API and may be wired in by a future caller without an `epsilon ≤ 1` guard.

Recommendation: add an `epsilon ≤ 1` guard in `GaussianMechanism::new`, or substitute the analytic Gaussian. Mark P0 if direct use is planned.

F-2 (Medium) Quartile sensitivity assumed equal to 1.0 and never enforced on input

Files: `aggregation/publish.rs:34, 219-227`

The constant `QUARTILE_SENSITIVITY = 1.0` assumes `claimed_value / 1000.0` lies in `[0, 1]`. The publication pipeline divides by 1000 but never clamps; if a tenant submits a `customer_stats` row with a `claimed_value > 1000`, the per-quartile L1 sensitivity rises proportionally and the calibrated Laplace noise no longer satisfies the advertised `(epsilon_per_metric, 0)`-DP guarantee for that metric.

The privacy notice exposed to the dashboard states "Sensitivity is fixed at 1.0 — operators must normalise upstream stats accordingly." This is correct as written, but it pushes the safety obligation onto the operator integration. A misconfigured operator silently degrades the privacy bound for every cohort using the affected metric.

Recommendation: Clamp values to `[0.0, 1.0]` in `publish_cohort` and `publish_cohort_with_ledger` before percentile computation, with a per-tenant warning when clipping is applied. Defense in depth; the upstream contract still holds.

F-3 (Medium) HE aggregator ciphertext validation is minimal

File: `aggregation/he_aggregator.rs:95-102`

`HeAggregator::add_encrypted` rejects only `ct.c == 0` and `ct.c >= n^2`. A malicious client can submit a ciphertext `c` with `gcd(c, n) != 1` (i.e., `c` not in $\mathbb{Z}_{n^2}^*$); decryption will not necessarily fail but the resulting plaintext is undefined. More importantly, since the homomorphism is over $\mathbb{Z}_n$, a client can submit `Enc(very large number)` to bias the aggregate without violating any check.

The source comment on line 91-94 documents the limitation and points to "customer signature / attestation" as the protocol-layer mitigation. We confirm the in-module check is correct as far as it goes, but it is **not** sufficient to defend against malicious submitters on its own.

Recommendation: At submission ingest, require either (a) a zero-knowledge range proof that the encrypted value lies in $[0, B]$ for a cohort-specific bound B (e.g., Boudot 2000, Lipmaa 2003, or a Bulletproofs-style construction adapted to Paillier message space), or (b) at minimum, server-side enforcement of $\text{gcd}(\text{ct}.c, n) == 1$ to reject ciphertexts that leak factorization. Range proofs are the soundness-correct path; the gcd check alone is not sufficient.

F-4 (Medium) Modular exponentiation in decryption is not constant-time

Files: `he/paillier.rs:263-277` (and all `modpow` call sites)

`num-bigint::modpow` is the standard Rust big-integer modular exponentiation; it is not constant-time and its execution time correlates with the bit pattern of the exponent. Decryption exponentiates by `lambda`, the Carmichael totient. Repeated timed decryption queries can in principle leak `lambda`, which is sufficient to factor `n`.

The deployment model (server-side decryption of cohort aggregates, with rate limiting, of cohort aggregates only and never per-customer plaintexts) makes the attack expensive in practice. We agree with the source comment recommending an HSM-resident decryption path for production. Provided the API rate-limit per client is enforced at the network edge, and decryption is only performed on the cohort aggregate (not per customer), the residual risk is acceptable for a compliance audit.

Recommendation: For the compliance pack, document the rate-limit policy and the operator authorization boundary in the same section that names HSM as the production path. Track HSM integration as a separate roadmap item (target: prior to first paying customer in a regulated vertical).

F-5 (Medium) RngCore generality places trust on caller

Files: every primitive accepts `&mut impl RngCore` rather than a CSPRNG-bound trait.

`LaplaceMechanism::add_noise`, `GaussianMechanism::add_noise`, `PaillierPublicKey::encrypt`, `sample_zn_star`, `gen_prime`, `miller_rabin`, and `HeAggregator::new` all accept any `RngCore`. Tests use `StdRng` (a deterministic seeded ChaCha20 implementation) which is statistically excellent but exposes the seed. Production correctness requires every call site to pass `rand::rngs::OsRng` or another CSPRNG.

We spot-checked `dp/ledger.rs::new_publication_id` which uses `OsRng` directly. The publication pipeline (`aggregation/publish.rs::publish_cohort_with_ledger`) takes `rng: &mut impl RngCore` as an argument, so the RNG choice is left to the caller (the HTTP handler layer).

Recommendation: Add a project-wide convention that production callers pass `OsRng`; add a doc-comment on each public RNG-accepting function naming the requirement; add a callsite audit as part of the production-readiness checklist.

F-6 (Low) Finite-precision floating-point sampling is theoretically subject to the Mironov 2012 attack

Files: `dp/laplace.rs:42-49`, `dp/gaussian.rs:51-65`

Both noise samplers use double-precision floating-point arithmetic. Mironov (CCS 2012) shows that finite-precision floating-point sampling of Laplace and Gaussian noise produces output distributions whose low-order bits leak information about the input. The attack requires the adversary to observe the precise low-order bits of the published noisy value.

In this codebase, the published quartiles are serialized as `f64` to JSON over HTTPS. An attacker who can observe the precise bit pattern (i.e., who reads the exact serialized double) could in principle execute the Mironov attack. However:

- The number of bits leaked per query is small (a handful of bits per noisy output).
- The k-anonymity gate ($k=10$ by default) and the per-cycle ϵ budget (`ledger.rs`) cap the number of usable queries per cohort.
- The published quartile is a function of 10+ contributors, so the marginal-distribution attack does not isolate one record.

The clamp in `laplace.rs:46` (`.min(1.0 - f64::EPSILON)`) further bounds the tail; combined with the lack of subnormal handling, the residual privacy parameter is effectively $(\epsilon, \sim 2^{-52})$ -DP rather than $(\epsilon, 0)$ -DP. The delta inflation is negligible for any practical compliance target (PII regulators accept $\delta \leq 1/N$ where N is the population; 2^{-52} is six orders of magnitude tighter than the typical $1e-6$ chosen here).

Recommendation: Document the residual $\delta \approx 2^{-52}$ in the compliance pack. Track integer-arithmetic sampling (e.g. Canonne-Kamath-Steinke 2020 discrete Gaussian, or the snapping mechanism) as a future hardening item.

F-7 (Low) Advanced composition path requires homogeneous epsilon

File: `dp/composition.rs:33-39`

`advanced_composition` rejects heterogeneous epsilon vectors. This is a strict interpretation of Theorem 3.20 (which is stated for homogeneous folds); the heterogeneous-fold extension by Kairouz-Oh-Viswanath (2015) is not implemented. This is a known limitation, not a bug, and is correctly enforced.

Recommendation: No action required unless heterogeneous composition is needed by a future cohort design.

F-8 (Low) Modulus length tolerance is plus or minus 2 bits

File: `he/paillier.rs:222-228` , test at `paillier.rs:563-566`

The test asserts $n_bits \in [bits - 2, bits + 2]$ for a requested 512-bit keypair. For 2048-bit production keys, the actual n may be 2046-2048 bits. The Paillier security reduction depends on the difficulty of the n -th-power residuosity problem under the modulus, which is governed by the size of the smaller of p and q . A 1-bit slop on the half-modulus translates to roughly halving the work-factor for the best known factoring attack. This is well within the security margin (NIST 2048-bit RSA targets 112-bit security; a one-bit reduction is operationally invisible).

Recommendation: No action required. If the operator wants to guarantee an exact bit-length, enforce `n.bits() == bits` in `from_primes`.

F-9 (Low) Per-quartile independent Laplace is correct but loose

File: `aggregation/publish.rs:194-244`

Releasing four quartiles per metric with `epsilon_per_metric / 4` Laplace noise on each and summing the budget via basic composition is provably `(epsilon_per_metric, 0)`-DP for the released vector. This is correct.

A tighter alternative would be smooth-sensitivity calibration (Nissim-Raskhodnikova-Smith 2007) or propose-test-release (Dwork-Lei 2009). For quartiles on a bounded `[0, 1]` range, the local sensitivity is typically much lower than the global sensitivity of 1, so smooth-sensitivity Laplace would give substantially less noise for the same epsilon. The trade-off is implementation complexity.

Recommendation: Track smooth-sensitivity quartiles as an accuracy improvement, not a correctness fix.

5. Positive Findings

The following items in the scope checklist were verified correct:

1. **Laplace inverse-CDF sampling** (`laplace.rs:42-49`) matches the standard form. The `(1.0 - f64::EPSILON)` clamp prevents `ln(0)` without distorting the bulk of the distribution; the delta cost is $\sim 2^{-52}$.
2. **Gaussian Box-Muller** (`gaussian.rs:59-65`) is correctly implemented. The unused `z1` second sample is wasteful but not incorrect.
3. **Sigma formula numerical check** (`gaussian.rs::sigma_formula` test) confirms `sigma` ≈ 4.84 for `epsilon = 1`, `delta = 1e-5`, `sensitivity = 1`.
4. **Basic composition** (`composition.rs:12-16`) sums epsilons and deltas — matches Dwork-Roth Thm 3.16 exactly.
5. **Advanced composition** (`composition.rs:21-45`) implements the formula `epsilon * sqrt(2k * ln(1/delta')) + k*epsilon*(e^epsilon - 1)` and delta `kdelta + delta'` from Dwork-Roth Thm 3.20.
6. **RDP accountant** (`composition.rs:62-102`) implements `RDP_alpha = alpha * sensitivity^2 / (2 * sigma^2)` for Gaussian (Mironov 2017 Prop 7) and converts to `(epsilon, delta)` via min over alpha of `RDP_alpha + ln(1/delta)/(alpha-1)` (Mironov 2017 Prop 3).
7. **Off-by-one in basic composition counter:** not present. `basic_composition` and the ledger both charge `N` publications when `N` are made.
8. **k-anonymity gate** (`k_anonymity.rs`, `publish.rs:202-216`) suppresses when contributor count is strictly less than `k`, where contributors are deduplicated per tenant by latest `submitted_at`. The "leak via different error messages" risk is addressed at the publication layer by surfacing the suppression reason only in the published metric envelope (k-anon vs budget exhaustion), not on a per-record path that could be used as an existence oracle.

9. **Ledger atomicity** (`ledger.rs::record_publication`) wraps the read-modify-write in `BEGIN IMMEDIATE TRANSACTION` on SQLite, which acquires a single-writer lock for the duration. Concurrent publications on the same (`cohort`, `metric`, `cycle_start`) row are serialized; no double-spend is possible on the SQLite backend. The Postgres backend is documented as requiring `SERIALIZABLE` isolation at the caller layer.
10. **Rotate semantics** (`ledger.rs::rotate_cycle`) cannot be triggered by an attacker — it is gated by `POST /v1/cohort/:id/budget/rotate`, an operator-only endpoint. Re-rotating to the same `cycle_start` does not zero the spend column (only the caps); the operator-error case is safe.
11. **Sequential composition tied to noise scale** (`publish.rs:194-244`): `per_quartile_eps = epsilon_per_metric / 4`; four independent Laplace draws at that scale; total charge per non-suppressed metric equals `epsilon_per_metric`. The ledger charge in `record_publication` matches: `cohort.epsilon_per_metric` is the full charge.
12. **Paillier key generation** (`paillier.rs:204-228`): two distinct primes of half-modulus length, Miller-Rabin with 40 rounds (industry standard), `n = pq`, `lambda = lcm(p-1, q-1)`, `g = n+1`, `mu = lambda^-1 mod n`. All correct per Paillier 1999.
13. **Encryption** (`paillier.rs:138-154`): the `(1 + m*n) * r^n mod n^2` form is the textbook simplification under `g = n + 1`. Confirmed by binomial expansion `(n+1)^m mod n^2 = 1 + mn`.
14. **Decryption** (`paillier.rs:263-277`): `u = c^lambda mod n^2`, `L(u) = (u-1)/n`, `m = L(u) * mu mod n`. Integer division in `L(u)` is exact because `u ≡ 1 mod n` by construction.
15. **Z_n vs Z_{n^2} arithmetic discipline**: all operations are explicitly reduced in the correct modulus. No leakage of `Z{n^2}` semantics into the plaintext space.
16. **Rejection sampling for r in Z_n*** (`paillier.rs:288-297`): rejection probability is `1/p + 1/q` which is negligible; 256 retries is overwhelming overkill.
17. **Signed fixed-point encoding** (`encoding.rs::encode_f64_for_modulus`): rejects magnitudes greater than or equal to `n/2`; the `>= u64::MAX as f64` overflow guard fires before the cast. Round-trip encode/decode is exact for in-range values.
18. **Homomorphic operations**:
 - `add(a, b) = a * b mod n^2` decrypts to `plain(a) + plain(b) mod n`. Confirmed by `test_homomorphic_add_decrypts_to_sum_mod_n` and `test_homomorphic_add_wraps_mod_n`.
 - `mul_scalar(a, k) = a^k mod n^2` decrypts to `plain(a) * k mod n`. Confirmed by `test_mul_scalar_decrypts_to_product_mod_n`.
 - `rerandomize(c, r')` produces a new ciphertext that decrypts to the same plaintext. Confirmed by `test_rerandomize_preserves_plaintext`.
19. **Acceptance property tests** requested in the scope (encrypt-decrypt identity, additive homomorphism, scalar multiplication) all pass at the boundary values `0`, `1`, `n/4`, `n/2 - 1`, `n - 1`.

6. Compliance Mapping

For an enterprise compliance pack (GDPR Article 25, EU AI Act Article 10, NIST SP 800-188):

Requirement	Mechanism	Status
Quantified disclosure risk	(epsilon, delta) -DP with per-cohort ϵ ledger	Implemented
Per-period release budgeting	dp/ledger.rs with BEGIN IMMEDIATE atomicity	Implemented
Re-identification gate	k-anonymity with default k=10	Implemented
Tamper-evident audit trail	Append-only dp_budget_publications table	Implemented
Encryption of in-flight contributions	Paillier with public-key envelope per cohort	Implemented
Server-side aggregation without per-customer plaintext access	HeAggregator	Implemented
Cryptographic agility (key rotation)	PEM-encoded keypair import/export	Implemented (operator-driven)
Side-channel resistance for decryption	Not implemented — see F-4	Pending (HSM integration)
Range proofs on encrypted submissions	Not implemented — see F-3	Pending (range-proof protocol)

7. Verdict (original, pre-remediation)

Acceptable with required fixes.

The implementation is faithful to the underlying primitives. The DP module correctly implements the Laplace mechanism, basic and advanced composition, RDP accounting, k-anonymity suppression, and per-cycle epsilon budgeting with atomic enforcement. The Paillier module is a textbook-correct implementation of the cryptosystem under the $g = n + 1$ simplification, including encryption, decryption, additive homomorphism, scalar multiplication, and re-randomization.

The findings in section 4 are either operating-envelope constraints (F-1, F-2, F-5), trust-boundary expansions that match the documented deployment model (F-4), or theoretical concerns with negligible practical exposure under the stated threat model (F-6, F-7, F-8, F-9). Finding F-3 (aggregator ciphertext validation) is the strongest substantive item; it is a known limitation acknowledged in the source comments and requires a protocol-level mitigation (range proofs) rather than a code fix in the audited modules.

The following remediations are required for the compliance pack to remain accurate when the code is deployed:

1. **(P0)** Add an `epsilon ≤ 1` guard in `GaussianMechanism::new`, or replace the formula with the analytic Gaussian mechanism. (F-1)
2. **(P0)** Clamp `claimed_value / 1000.0` to `[0, 1]` in `publish.rs` before percentile computation, with operator visibility on clipped tenants. (F-2)

3. **(P1)** Specify range-proof protocol for HE submissions; until then, document the trust assumption on submitters in the compliance pack. (F-3)
4. **(P1)** Integrate HSM-backed Paillier private-key storage and decryption for production deployments. (F-4)
5. **(P1)** Audit every call site of the RNG-accepting public functions and confirm `0sRng` is used in production paths. (F-5)
6. **(P2)** Document the floating-point delta inflation ($\sim 2^{-52}$) in the privacy notice surfaced to operators. (F-6)

The remaining findings (F-7, F-8, F-9) are tracked for future accuracy improvements but do not block compliance certification under the threat model stated in section 3.

8. Post-review remediation (addendum, 2026-05-25)

Following the original review, the engineering team applied code-level remediations on the same date. This section documents which findings were closed in code, which remain open, and the test evidence supporting closure. The original findings in section 4 are preserved as historical record.

8.1 Closed in code

Finding	Status	Remediation	File(s)	New tests
F-1 (Medium-High)	Closed	<code>GaussianMechanism::new</code> now rejects $\epsilon > 1$ with <code>DpError::InvalidEpsilon</code> . New constant <code>MAX_GAUSSIAN_EPSILON = 1.0</code> documents the operating envelope.	<code>core/src/dp/gaussian.rs</code>	<code>rejects_epsilon_a</code>
F-2 (Medium)	Closed	New <code>clamp_unit()</code> helper bounds each per-tenant value to <code>[0, 1]</code> before percentile computation in both <code>publish_cohort</code> and <code>publish_cohort_with_ledger</code> . NaN maps to 0.0 so the DP guarantee cannot be defeated by malformed input. Privacy notice surfaced to operators now explicitly states the clamp.	<code>core/src/aggregation/publish.rs</code>	<code>clamp_unit_bounds</code> <code>quartile_sensitivity</code>
F-3 (partial — gcd hardening)	Tightened (full closure pending range proofs)	<code>HeAggregator::add_encrypted</code> now rejects ciphertexts with <code>gcd(c, n) ≠ 1</code> . Defends against factorisation-leaking ciphertexts; does not defend against malicious-large-plaintext griefing. The docstring explicitly states the residual gap.	<code>core/src/aggregation/he_aggregator.rs</code>	<code>test_aggregator_r</code>
F-5 (Medium)	Closed (documentation)	Every public RNG-accepting function now carries a <code># RNG requirement</code> doc-section explicitly naming <code>rand::rngs::0sRng</code> as the	<code>core/src/dp/laplace.rs</code> , <code>core/src/dp/gaussian.rs</code> ,	n/a (doc-only)

Finding	Status	Remediation	File(s)	New tests
		production-required CSPRNG. Affected: LaplaceMechanism::add_noise, GaussianMechanism::add_noise, PaillierPublicKey::encrypt, PaillierPublicKey::rerandomize, PaillierPrivateKey::generate, HeAggregator::new, publish_cohort, publish_cohort_with_ledger. Call-site audit remains an operator obligation.	core/src/he/paillier.rs, core/src/aggregation/he_aggregator.rs, core/src/aggregation/publish.rs	
F-6 (Low)	Closed (documentation)	Laplace module docs and both publication privacy notices now explicitly state that floating-point sampling produces $(\epsilon, \sim 2^{-52})$ -DP rather than $(\epsilon, 0)$ -DP, and that the inflation is negligible against the chosen $\delta = 1e-6$.	core/src/dp/laplace.rs, core/src/aggregation/publish.rs	n/a (doc-only)

8.2 Open

Finding	Status	Why deferred
F-3 (range proofs)	Open, tracked	Full mitigation requires a protocol-layer zero-knowledge range proof on every encrypted submission (Boudot 2000, Lipmaa 2003, or a Bulletproofs-style construction adapted to Paillier message space). This is a design decision outside the audited module surface. The gcd hardening above is a strict improvement, not a substitute.
F-4 (HSM integration)	Open, tracked	Requires production infrastructure work (HSM provisioning, key-management envelope protocol, deployment pipeline). The documented deployment-model mitigations (server-side decryption only, rate-limiting on the decrypt path, cohort-aggregate-only decryption — never per-customer plaintext) remain in force in the interim.
F-7, F-8, F-9	Open, accuracy-improvement	Already classified as Low at original review. No code change required for compliance certification; tracked as future accuracy/optimisation work.

8.3 Test evidence after remediation

Total in-tree unit-test count for the audited modules increased from 72 to 76:

Module group	Before	After	Delta
dp::*	32	33	+1 (F-1)
he::*	28	28	0
aggregation::publish	9	11	+2 (F-2)

Module group	Before	After	Delta
aggregation::he_aggregator	3	4	+1 (F-3)
Total	72	76	+4

All 76 tests pass under `cargo test --lib --quiet`. `cargo check --quiet` reports 0 errors and 0 warnings in any of the audited files (one unrelated `unused_mut` warning in `core/src/bin/sauronid-cli.rs`).

8.4 Updated verdict

Acceptable for the documented deployment model.

P0 findings (F-1, F-2) are closed in code with new regression tests. P1 finding F-5 is closed via documentation; the operator-side call-site audit is the remaining obligation and is in the scope of the production-readiness checklist. P1 finding F-3 is partially closed (factorisation-leak surface eliminated); full closure requires range proofs at the submission protocol layer and is tracked. P1 finding F-4 is deferred to infrastructure work; the documented compensating controls (server-side decryption only, rate-limiting, cohort-aggregate-only access) remain in force. Low-severity findings (F-6, F-7, F-8, F-9) are documented or tracked.

The implementation as of this addendum is suitable for production deployment under: - the threat model stated in section 3, - the documented compensating controls for F-4 (HSM-deferred), - a submission-layer trust assumption on cohort participants pending the range-proof rollout for F-3.

This addendum does **not** require a fresh independent review unless the audited module surface changes again. Material changes to `core/src/dp/`, `core/src/he/`, `core/src/aggregation/publish.rs`, or `core/src/aggregation/he_aggregator.rs` invalidate the addendum and warrant a new review.

Signature:

Reviewer: _____

Credential: _____

Date: _____

SauronID counter-signature (post-remediation):

Engineering lead: _____

Date: _____